Solution: Maximum Sum Subarray of Size K

Problem Statement

Given an array of positive numbers and a positive number 'k,' find the **maximum sum of any contiguous subarray of size** 'k'.

Example 1:

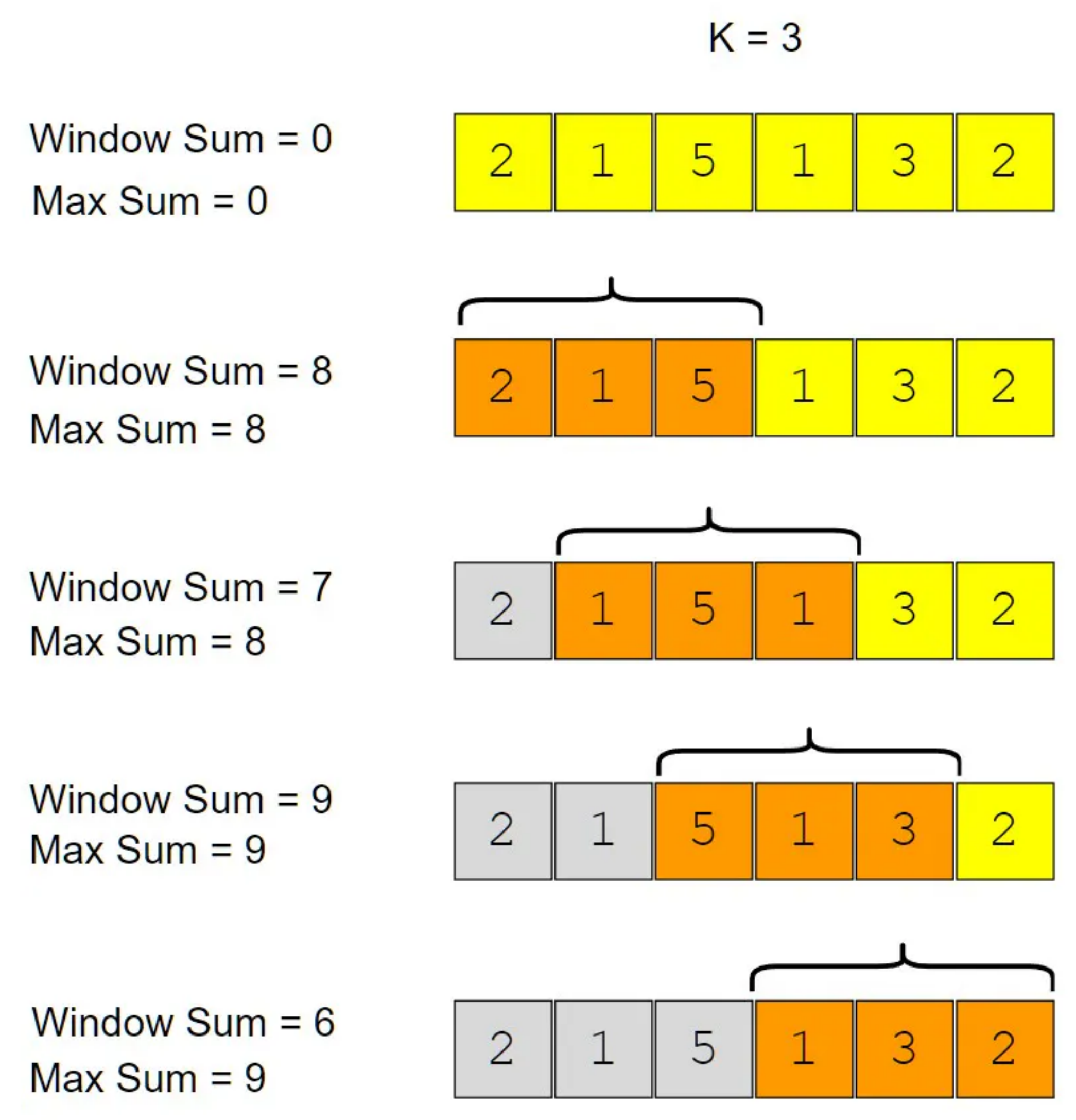
```
Input: [2, 1, 5, 1, 3, 2], k=3
Output: 9
Explanation: Subarray with maximum sum is [5, 1, 3].
```

Example 2:

```
Input: [2, 3, 4, 1, 5], k=2
Output: 7
Explanation: Subarray with maximum sum is [3, 4].
```

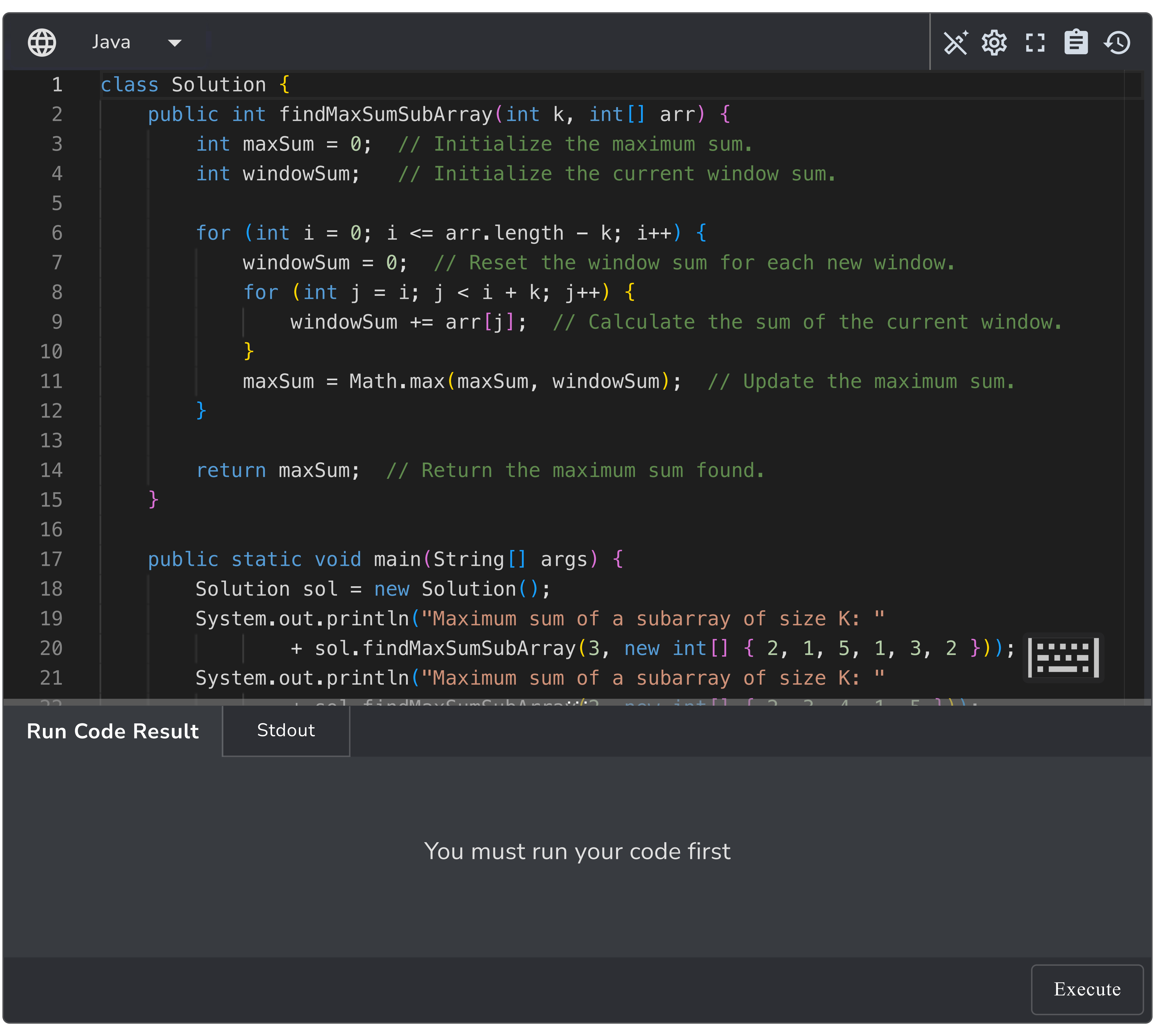
Solution

A basic brute force solution will be to calculate the sum of all 'k' sized subarrays of the given array to find the subarray with the highest sum. We can start from every index of the given array and add the next 'k' elements to find the subarray's sum. Following is the visual representation of this algorithm for Example-1:



Code

Here is what our algorithm will look like:



```java
class Solution {
    public int findMaxSumSubArray(int k, int[] arr) {
        int maxSum = 0;  // Initialize the maximum sum.
        int windowSum;   // Initialize the current window sum.

        for (int i = 0; i <= arr.length - k; i++) {
            windowSum = 0;  // Reset the window sum for each new window.
            for (int j = i; j < i + k; j++) {
                windowSum += arr[j];  // Calculate the sum of the current window.
            }
            maxSum = Math.max(maxSum, windowSum);  // Update the maximum sum.
        }

        return maxSum;  // Return the maximum sum found.
    }

    public static void main(String[] args) {
        Solution sol = new Solution();
        System.out.println("Maximum sum of a subarray of size K: "
                + sol.findMaxSumSubArray(3, new int[] { 2, 1, 5, 1, 3, 2 }));
        System.out.println("Maximum sum of a subarray of size K: "
```

You must run your code first

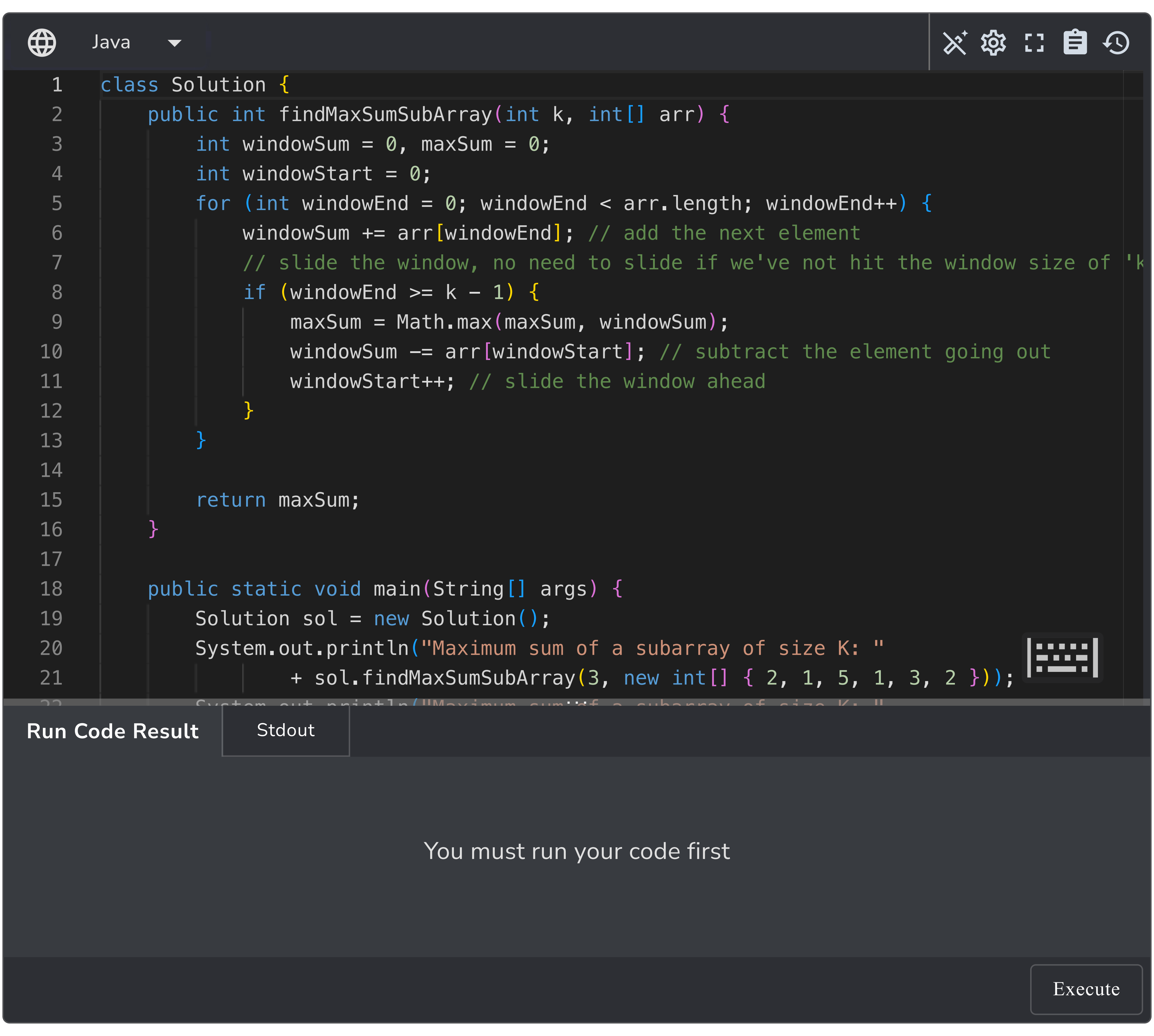
The above algorithm's time complexity will be $O(N * K)$, where 'N' is the total number of elements in the given array. Is it possible to find a better algorithm than this?

A Better Approach

If you observe closely, you will realize that to calculate the sum of a contiguous subarray, we can utilize the sum of the previous subarray. For this, consider each subarray as a **Sliding Window** of size 'k.' To calculate the sum of the next subarray, we need to slide the window ahead by one element. So to slide the window forward and calculate the sum of the new position of the sliding window, we need to do two things:

1. Subtract the element going out of the sliding window, i.e., subtract the first element of the window.
2. Add the new element getting included in the sliding window, i.e., the element coming right after the end of the window.

This approach will save us from re-calculating the sum of the overlapping part of the sliding window. Here is what our algorithm will look like:



```java
class Solution {
    public int findMaxSumSubArray(int k, int[] arr) {
        int windowSum = 0, maxSum = 0;
        int windowStart = 0;
        for (int windowEnd = 0; windowEnd < arr.length; windowEnd++) {
            windowSum += arr[windowEnd]; // add the next element
            // slide the window, no need to slide if we've not hit the window size of 'k
            if (windowEnd >= k - 1) {
                maxSum = Math.max(maxSum, windowSum);
                windowSum -= arr[windowStart]; // subtract the element going out
                windowStart++; // slide the window ahead
            }
        }

        return maxSum;
    }

    public static void main(String[] args) {
        Solution sol = new Solution();
        System.out.println("Maximum sum of a subarray of size K: "
                + sol.findMaxSumSubArray(3, new int[] { 2, 1, 5, 1, 3, 2 }));
```

You must run your code first

Time Complexity

The time complexity of the above algorithm will be $O(N)$.

Space Complexity

The algorithm runs in constant space $O(1)$.

← Previous
Maximum Sum Subarray Of Size K (Easy)

Next →
Smallest Subarray With A Greater Sum (Easy)

☐ Mark as Completed